

Introduction to Retrieval-Augmented Generation (RAG)

Subtitle: Enhancing Language Models with External Knowledge

👨‍🏫 **Presented by:** Anis Koubaa

📅 **Date:** April 08, 2025

📘 **Course:** SE445: Generative AI

 **by Anis Koubaa**

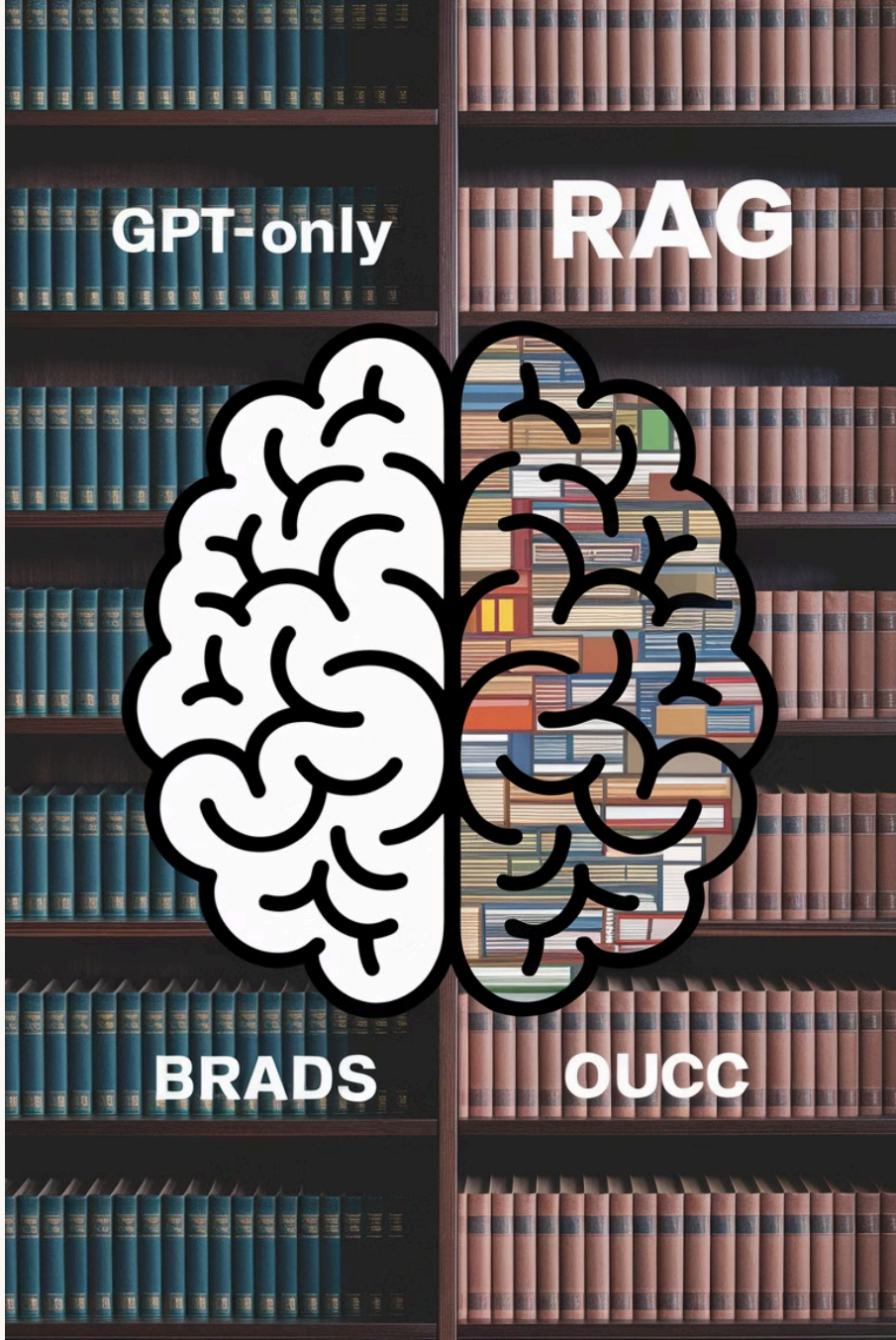
What's Wrong with Plain Old GPT?

The Problem with Traditional LLMs:

- Rely solely on **pre-trained static knowledge**.
- Lack access to **real-time**, **private**, or **domain-specific info**.
- Prone to **hallucination**: confident but incorrect answers.

RAG Solution:

Dynamically pulls relevant info from external sources at runtime.



Limitations of Fine-Tuning Language Models

🔒 1. Static Knowledge

- Model remains **frozen** with training data
- Unable to incorporate **new information**

💰 2. High Cost

- Demands significant **compute resources**
- Expensive for large models with billions of parameters

🧠 3. Catastrophic Forgetting

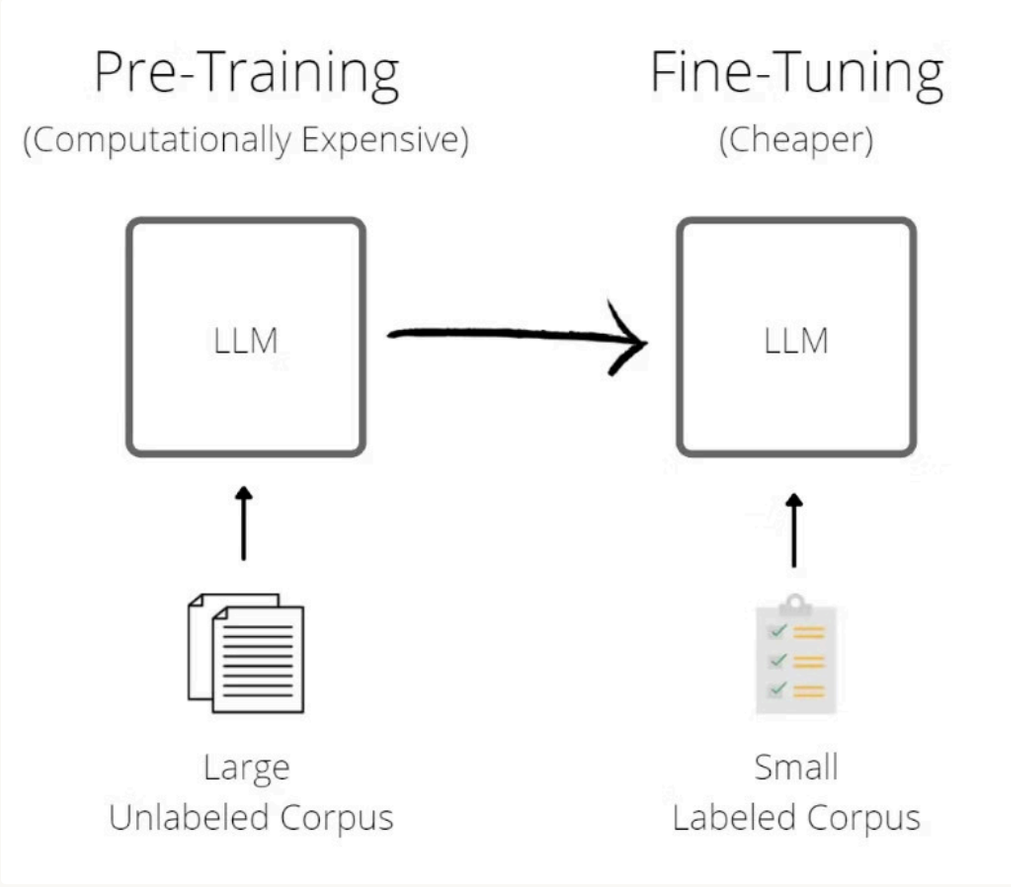
- Domain-specific training may **erase general knowledge**

🧱 4. Lack of Modularity

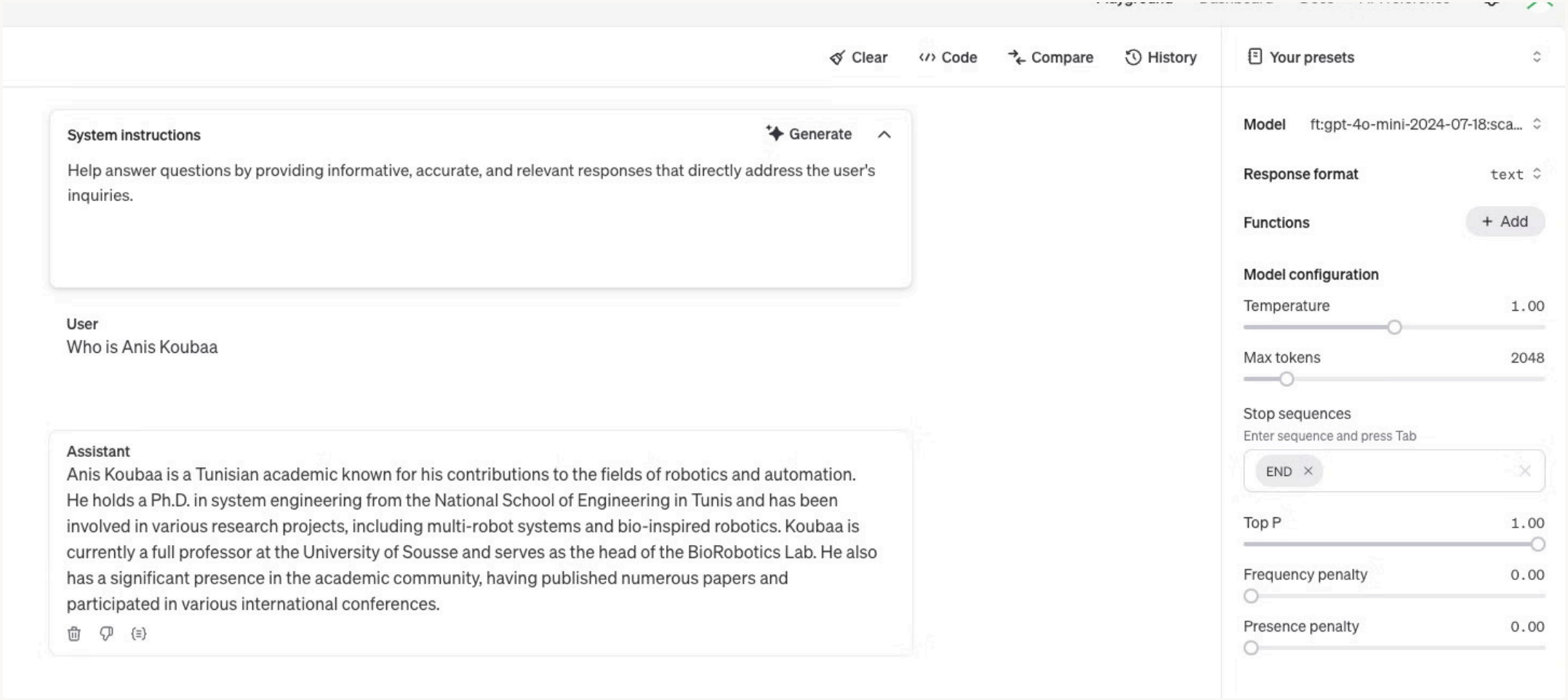
- Cannot update specific components without **retraining** everything

🔬 5. Evaluation & Debugging

- Difficult to pinpoint **what changed** and **why**
- Errors become embedded and hard to trace



Hallucination in Fine-Tuned Models



RAG in a Nutshell — Blending Search and Generation

What is RAG?

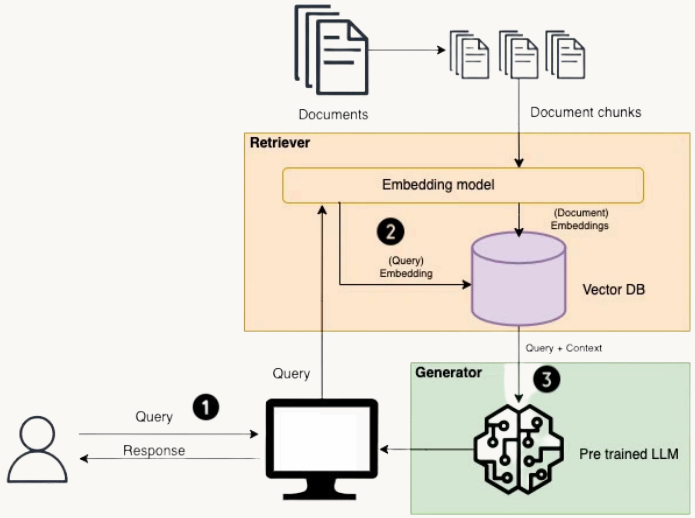
RAG is a hybrid architecture that **combines document retrieval with generative AI**.

Retrieval brings **external, up-to-date knowledge**, and generation **crafts a natural response**.

🧠 "Retrieval" fetches facts → "Generation" produces responses.

Why It Matters:

Boosts **accuracy**, **relevance**, and **factual grounding** in AI answers.



Behind the Magic — How RAG Actually Works

Retrieval

Find relevant documents or passages.

Documents

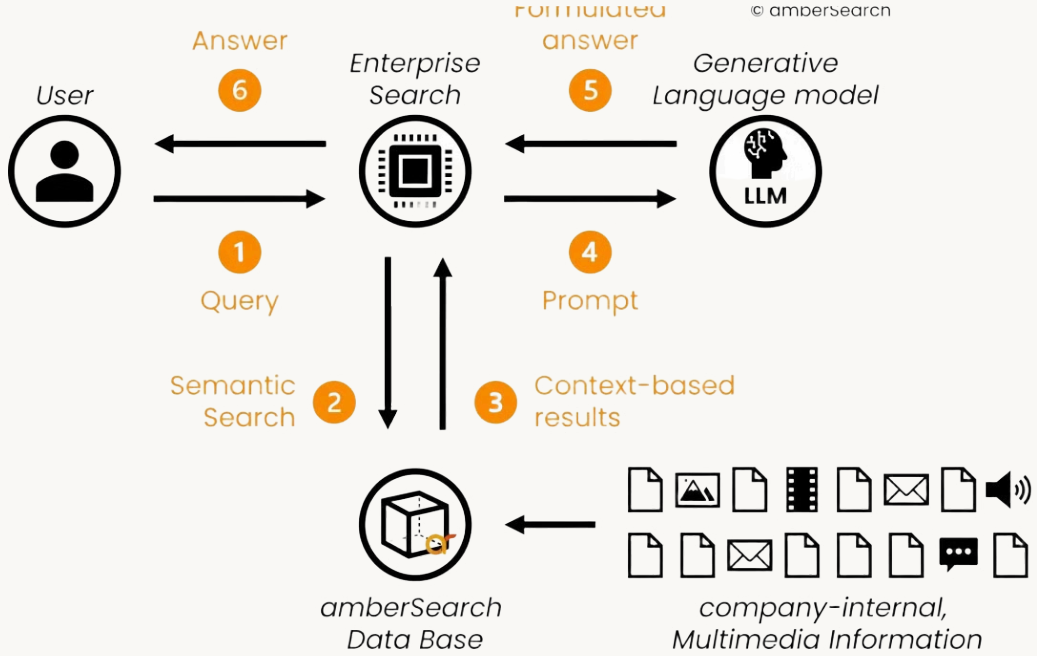
Relevant information is collected.

Generation

Use LLM to create a response based on retrieved data.

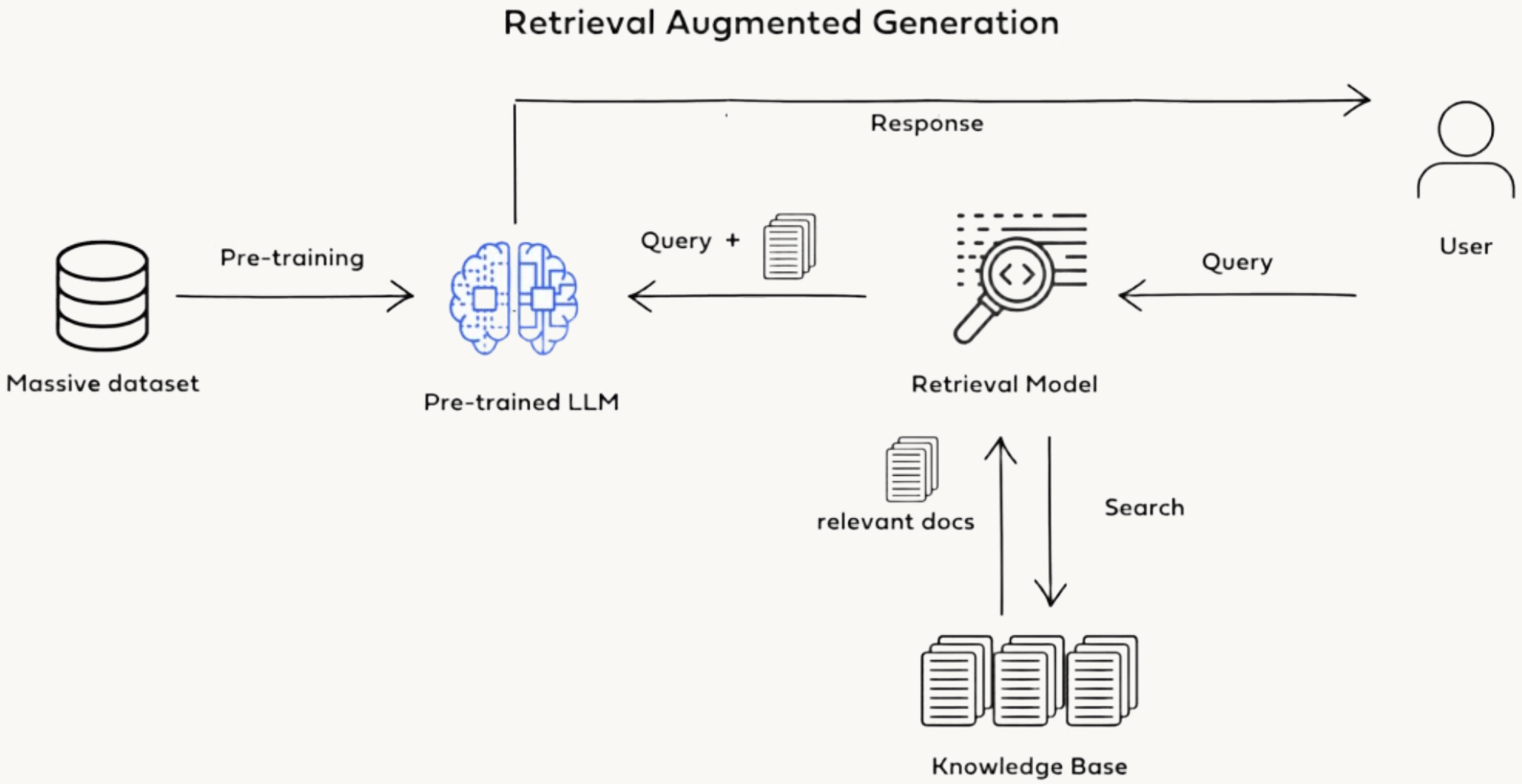
Final Answer

Accurate, grounded response.



🧑‍🎓 *Analogy:* Open Book Exam

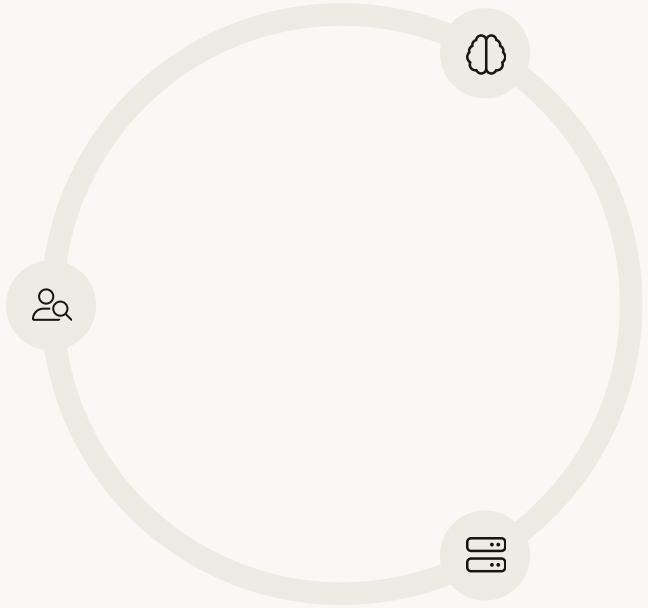
RAG: The Big Picture



Under the Hood — RAG System Components

🔍 Retriever

Finds relevant documents (e.g., BM25, DPR).



🧠 Generator

Language model (e.g., GPT, T5) for crafting output.

📚 Knowledge Base

Indexed external data (e.g., PDFs, websites).

Integration: Retrieved texts are **injected as context** into the generator.



🔍 The Retrieval Step — Deep Dive

🧮 Convert to Embeddings

Converts queries & docs into **dense embeddings** (numerical vectors).

📏 Measure Similarity

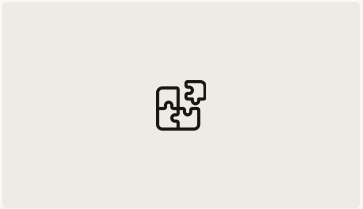
Measures similarity (e.g., **cosine similarity**) between vectors.

📄 Return Results

Returns **Top-k most relevant** documents.

🛠️ **Tools/Models:** DPR, Sentence Transformers, FAISS, Chroma

The Generation Step — Deep Dive



Combine Input

Input = Query + Top-k docs → Combined into a single context.



Process Context

LLM generates answer **conditioned on this context**.

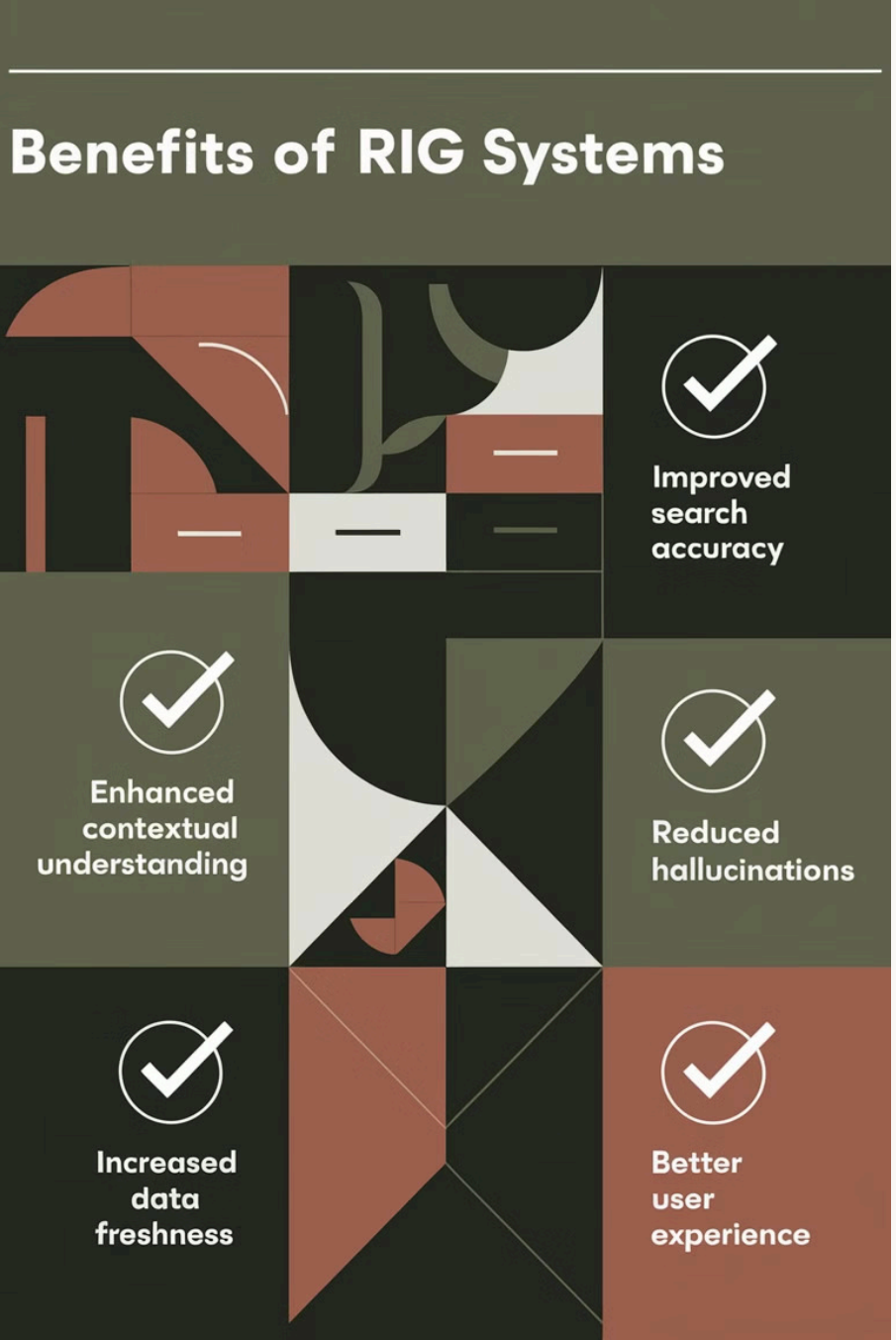


Fine-tune Output

Can be fine-tuned for specific use cases (Q&A, summarization).

Example:

- Query: *"What's the capital of Brazil?"*
- Retrieved: *"Brazil's capital is Brasília."*
- Generated: "The capital of Brazil is Brasília."



Why Use RAG? Benefits at a Glance

Up-to-Date Knowledge

No more outdated responses.

Reduces Hallucinations

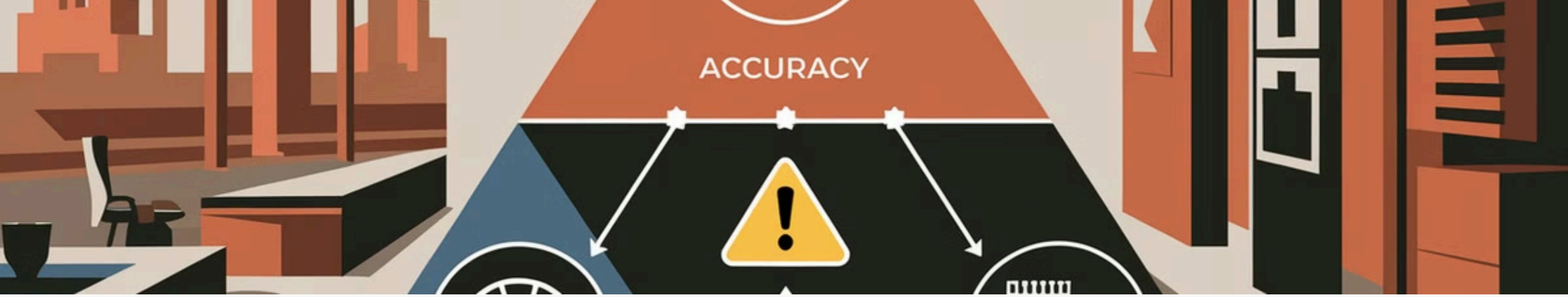
Grounds answers in retrieved facts.

Domain Flexibility

Works with any custom data source.

Scalability

Update KB without retraining the model.



Challenges and Limitations

🔍 **Retriever Quality**	Garbage in = Garbage out.
⏳ **Latency & Cost**	Slower than LLM-only due to 2-step process.
📏 **Input Size Limits**	Long documents may be truncated.
📐 **Evaluation Complexity**	Hard to assess retrieval + generation quality.

RAG is powerful but not perfect.

Applications & What's Next

Customer Support

🤖 Customer support assistants (e.g., airline chatbots)

Research Q&A

👩‍🏫 Research Q&A systems

Fact-Checking

📰 Real-time fact-checkers

Future Directions:

- Multi-modal RAG (text + images)
- Smarter retrievers (context-aware, intent-based)
- Memory + caching for faster responses

Closing Message:

RAG bridges **static AI** and **dynamic knowledge**. Expect it to redefine how AI systems learn and respond!



Unlocking Smarter AI: Introduction to Retrieval-Augmented Generation (RAG)

Subtitle: Introduction to Vectorstores

👨‍🏫 **Presented by:** Anis Koubaa

📅 **Date:** April 06, 2025

📘 **Course:** SE445: Generative AI

 **by Anis Koubaa**

What is a Vector Store?

🗂️ Definition

A specialized database optimized for storing and searching **high-dimensional vector embeddings**.

📐 Why It's Important for RAG

- Enables **dense retrieval** through vector similarity comparison.
- Stores vectorized document representations.
- Retrieves **Top-k** similar items based on query vectors.

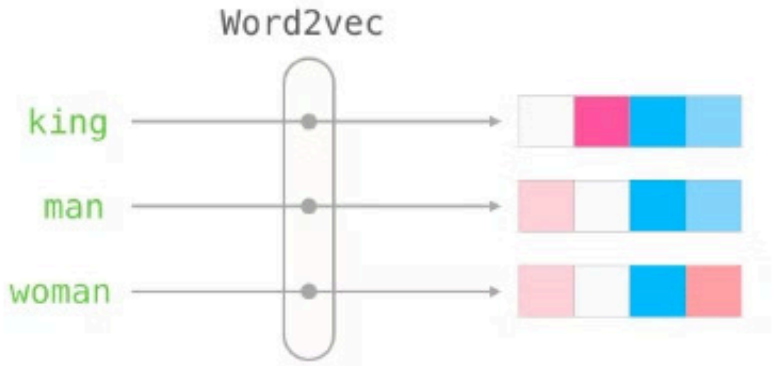
🧮 How It Works

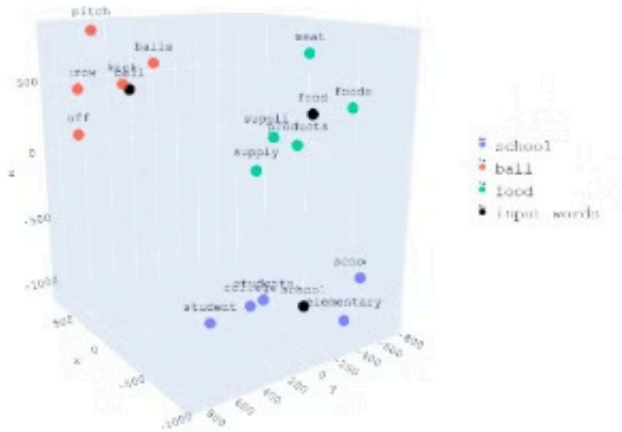
1. Convert documents to **embeddings** using encoder models.
2. **Index** these vectors efficiently.
3. **Transform queries** into the same vector space.
4. Return **relevant documents** based on similarity.

🔍 Example Use

> PDF → Embeddings → FAISS index → Query embedding → Top-5 chunks

From Token to Vectors - Embeddings






```
[ 0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 ,
-0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961
, -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 ,
-0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 ,
-1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042 ]
```

http://jalammar.github.io/illustrated-word2vec/

Semantic Search Engine and Vector Database, with Python, NumPy, and Word Embeddings
https://www.youtube.com/watch?v=Dx_znrWj38g

Convert Words to Embeddings

```python
import spacy
import numpy as np

# Load the small English model in spaCy
nlp = spacy.load("en_core_web_sm")

# Choose a word and convert it to a vector using spaCy
word = "apple"
doc = nlp(word)
embedding_vector = doc.vector

# Convert the spaCy vector to a NumPy array
embedding_vector = np.array(embedding_vector)

# Print the embedding vector
print(f"Embedding vector for the word '{word}':")
print(embedding_vector)

#Print the Shape of the vector
print(f"Shape of the embedding vector for the word '{word}' is {embedding_vector.shape}:")
```

```
Embedding vector for the word 'apple':
[-1.2599487  -0.87038326 -1.0834986   0.5798379   0.03857595 -0.02588724
  0.9775548   0.31093895  0.19412561 -0.8062178   0.43808204 -1.8498268
 -0.30574286  0.5693637   0.42844394  0.69174224 -0.7197368  -1.2614795
  0.83457553  0.14667332  0.12171662  0.48029226  0.50147873 -0.4299112
  0.5533447   0.8749714   0.71914     0.5731143  -0.5064311   0.38493997
 -0.31778833  0.18084693  0.5162936  -0.00233826  0.1870515  -1.3773322
  1.009095   -0.10771251  1.6994228   0.78603184 -0.8166558   0.57896584
 -0.5232718   0.7045958  -0.46308953 -0.37629813 -0.38788998  0.1730735
 -0.05550597 -0.17245518  0.62919456  0.87473     0.60047954 -0.27686393
  0.8524152  -0.28676936  0.9972549  -0.71060055  0.11830124 -0.37214422
 -1.3039289  -0.02281845  0.4063236  -0.43118405  0.9401908  -0.02761412
 -0.39026427 -0.29733896  0.78710043 -0.34422576  0.11906591  0.8003473
  1.4978364  -0.38792044 -0.5264353  -0.38889915 -0.28553864 -0.22295064
  0.8420893  -0.79365766 -0.0956156  -0.96640915 -1.1665895  -1.1019065
 -0.602306    0.765056    0.3859367  -0.31349194 -1.237845    0.11333084
 -1.6053262   0.14791119  0.6127024   1.0456864   0.8747115   0.6120273 ]
Shape of the embedding vector for the word 'apple' is (96,):
```

Explanation:

1. We use the `en_core_web_sm` model from spaCy, which is a small pre-trained model for English.
2. For a word like `apple`, we get the word's vector (embedding) using spaCy's `doc.vector`.
3. We print the embedding vector and demonstrate how to compute the similarity (in this case, using the dot product) between two words (`apple` and `banana`).

Notes:

- The `en_core_web_sm` model uses relatively simple embeddings, but larger models (like `en_core_web_md` or `en_core_web_lg`) provide richer embeddings with more detailed word representations.
- If you want to use more advanced embeddings, you can download larger models or use pre-trained models like GloVe or Word2Vec.

Cosine Similarity: Similarity of two Words

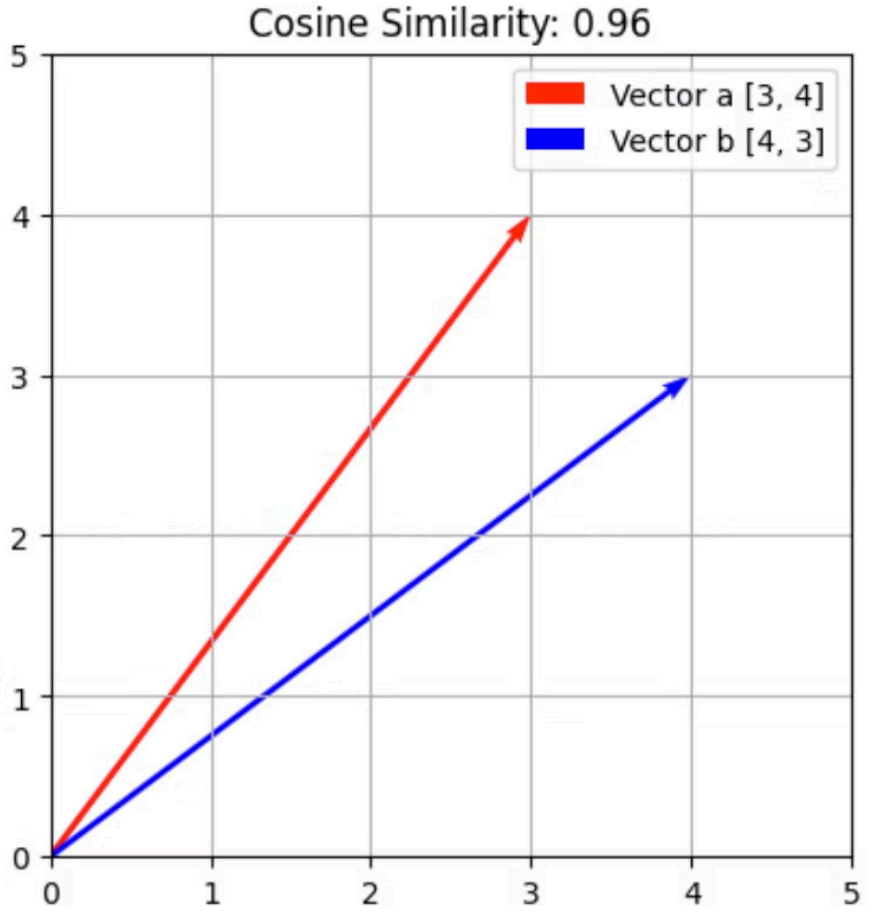
- **Definition:**
  - Cosine similarity is a measure of similarity between two non-zero vectors within an inner product space that measures the cosine of the angle between them.
- **Math Equation:**
  - **Compact Form:**

$$\text{Cosine Similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

  - **Expanded Form:**
    - Given vectors $\mathbf{x} = [x_1, x_2, \ldots, x_n]$ and $\mathbf{y} = [y_1, y_2, \ldots, y_n]$:

$$\text{Cosine Similarity} = \frac{x_1 y_1 + x_2 y_2 + \ldots + x_n y_n}{\sqrt{x_1^2 + x_2^2 + \ldots + x_n^2} \cdot \sqrt{y_1^2 + y_2^2 + \ldots + y_n^2}}$$

- **Purpose:**
  - **Similarity Measurement:**
    - Cosine similarity is widely used in various applications such as text analysis, where it helps in identifying the similarity between documents. It is especially useful in high-dimensional positive spaces like text data in TF-IDF representation.
    - In machine learning, it helps in clustering and classification by measuring how similar the data objects are, irrespective of their magnitude.



Cosine Similarity: Similarity of two Words

```python
import numpy as np
import matplotlib.pyplot as plt

# Define the vectors
a = np.array([3, 4])
b = np.array([4, 3])

# Calculate the dot product
dot_product = np.dot(a, b)

# Calculate the norms of the vectors
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)

# Calculate the cosine similarity
cosine_similarity = dot_product / (norm_a * norm_b)

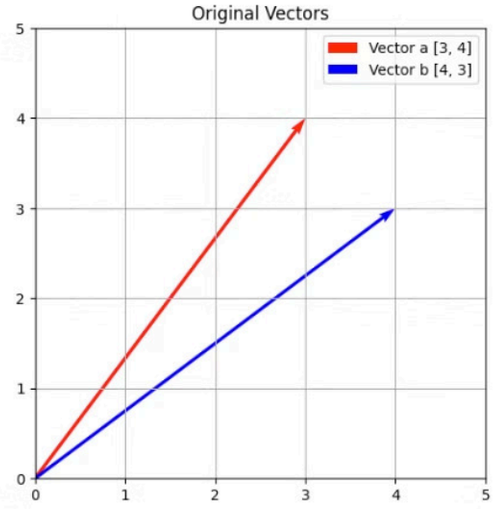
print("Cosine Similarity:", cosine_similarity)
```

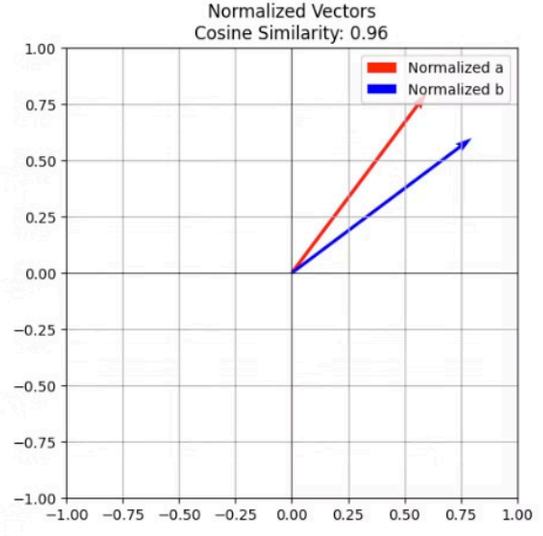
Numerical Example:

Consider two vectors in a 2D space:

- $\mathbf{a} = [3, 4]$
- $\mathbf{b} = [4, 3]$

$$\text{Cosine Similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$





Cosine Similarity: Similarity of two Words

```python
import spacy
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_lg")

# Define the words
words = ["apple", "banana", "fruit", "lion", "tiger", "leopard"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

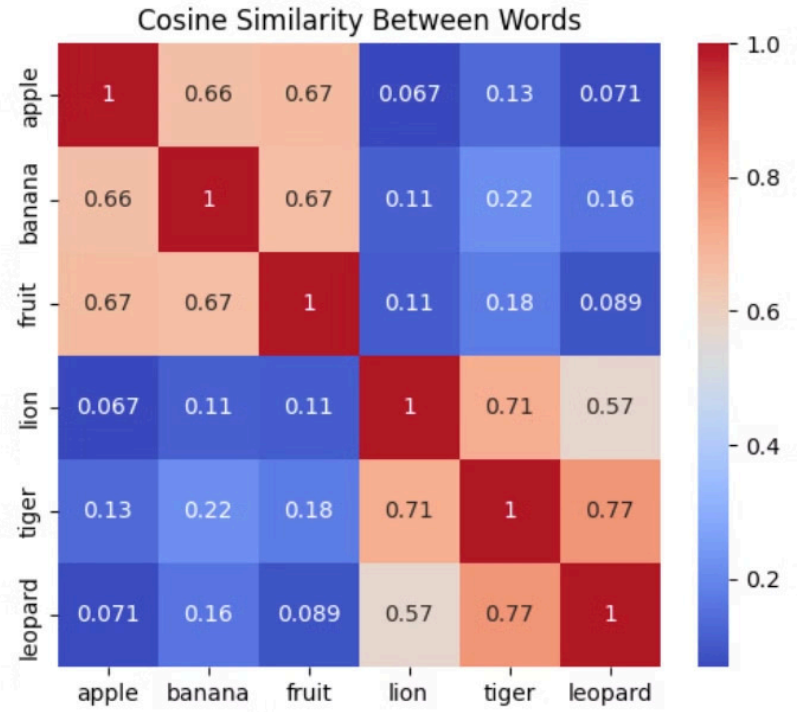
# Function to normalize a vector
def normalize_vector(vector):
    norm = np.linalg.norm(vector)  # Calculate the L2 norm
    if norm == 0:  # To prevent division by zero
        return vector
    return vector / norm

# Normalize the embeddings
normalized_embeddings = {word: normalize_vector(vector) for word, vector in embeddings.items()}

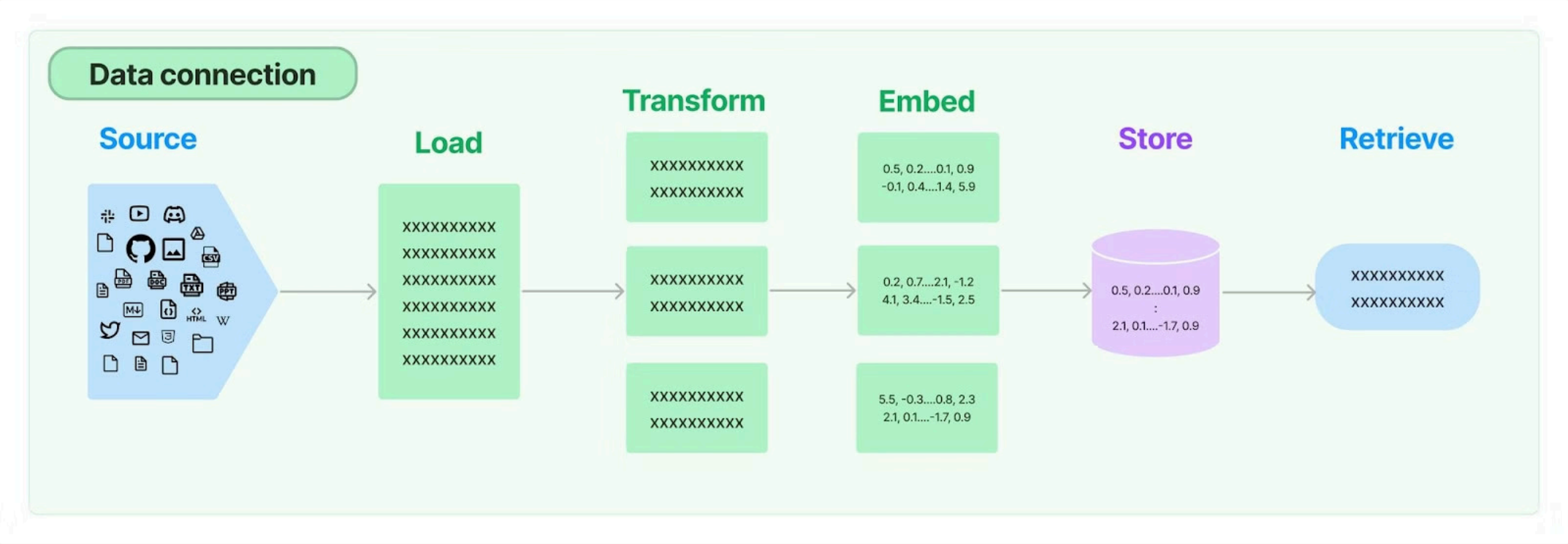
# Calculate pairwise cosine similarities
cosine_similarities = np.zeros((len(words), len(words)))

for i, word1 in enumerate(words):
    for j, word2 in enumerate(words):
        cosine_similarities[i, j] = np.dot(normalized_embeddings[word1], normalized_embeddings[word2])

# Plotting the cosine similarities using a heatmap
plt.figure(figsize=(6, 5))
sns.heatmap(cosine_similarities, annot=True, cmap='coolwarm', xticklabels=words, yticklabels=words)
plt.title('Cosine Similarity Between Words')
plt.show()
```



Vector Store or Vector Database



Tools & Features of Vector Stores

🧰 Popular Vector Store Libraries

- **FAISS** (by Facebook): Fast, flexible, open-source.
- **Chroma**: Simple Python-native DB for LLM apps.
- **Weaviate**: Full-featured with REST API and hybrid search.
- **Pinecone**: Scalable managed service.
- **Qdrant**, **Milvus**: Open-source, optimized for speed and scale.

⚙️ Key Features

Feature	Description
Indexing	Organizes vectors for efficient retrieval
Similarity Search	Cosine, L2, inner product
Metadata Filtering	Retrieve by tags (e.g., source, topic)
Persistence	Keeps data between sessions
Scalability	Handles millions of vectors

Embedding Techniques

🧬 What are Embeddings?

Embeddings are **numerical vector representations** of text that capture **semantic meaning**.

🧠 Why Use Embeddings in RAG?

- Convert both documents and queries into the same **vector space**.
- Enable **semantic similarity** search (beyond keyword matching).
- Foundation of **dense retrieval**.

📏 Similarity Metrics

- **Cosine Similarity** (most common)
- **Dot Product**, **Euclidean Distance**

🛠️ Popular Embedding Models

Model	Description
Sentence-BERT	Captures sentence-level meaning
OpenAI Embeddings	Used in GPT APIs
Instructor-XL	Instruction-tuned embeddings
Cohere, Hugging Face	Commercial & open-source options

Embedding Example in Python

```python
from openai import OpenAI


# Function to get embeddings
def get_embedding(text, model="text-embedding-ada-002"):
    client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))
    response = response = client.embeddings.create(
        model=model,
        input=text,
        encoding_format="float"
    )
    embedding = response.data[0].embedding
    return embedding

# Example usage
model="text-embedding-ada-002"
text="Hello, world!"
embedding = get_embedding(text, model)
embedding

```

```
-0.018009787,
0.007860908,
-0.007586096,
0.0064037642,
0.025218816,
-0.016373696,
0.0025324267,
0.017639112,
```

Chroma DB Vectorstore Example

```python
import os
from openai import OpenAI
from chromadb import Client
from chromadb.utils import embedding_functions

def create_vector_db():
    # Initialize ChromaDB and OpenAI embedding function
    chroma_client = Client()
    openai_ef = embedding_functions.OpenAIEmbeddingFunction(
        api_key=os.getenv("OPENAI_API_KEY"),
        model_name="text-embedding-3-small"
    )

    # Create collection
    collection = chroma_client.create_collection(
        name="documents",
        embedding_function=openai_ef
    )

    # Sample documents
    documents = [
        "The sky is blue because of Rayleigh scattering of sunlight.",
        "Python is a popular programming language created by Guido van Rossum.",
        "Machine learning is a subset of artificial intelligence.",
        "Neural networks are inspired by biological brains."
    ]

    # Add documents to collection
    collection.add(
        documents=documents,
        ids=[f"doc_{i}" for i in range(len(documents))]
    )

    return collection
```

```python
# Get all documents and their embeddings
results = collection.get(
    include=['documents', 'embeddings']
)

print("Documents in the database:")
for i, (doc, embedding) in enumerate(zip(results['documents'], results['embeddings'])):
    print(f"\nDocument {i}:")
    print(f"Text: {doc}")
    print(f"Embedding dimensions: {len(embedding)}")
    print(f"First 5 embedding values: {embedding[:5]}")

# Get collection stats
print(f"\nTotal documents: {len(results['documents'])}")
```

```
Documents in the database:

Document 0:
Text: The sky is blue because of Rayleigh scattering of sunlight.
Embedding dimensions: 1536
First 5 embedding values: [0.02326874 0.01073772 0.00854036 0.02833872 0.03781447]

Document 1:
Text: Python is a popular programming language created by Guido van Rossum.
Embedding dimensions: 1536
First 5 embedding values: [ 0.02183548 -0.0353007   0.01820581  0.00841337  0.01055382]

Document 2:
Text: Machine learning is a subset of artificial intelligence.
Embedding dimensions: 1536
First 5 embedding values: [-0.02127955 -0.05323635 -0.01303747 -0.02886602  0.05552166]

Document 3:
Text: Neural networks are inspired by biological brains.
Embedding dimensions: 1536
First 5 embedding values: [-0.00888277  0.0037943  -0.02780523  0.035786    0.01890285]

Total documents: 4
```



Unlocking Smarter AI: Introduction to Retrieval-Augmented Generation (RAG)

Subtitle: Chunking Techniques

👨‍🏫 **Presented by:** Anis Koubaa

📅 **Date:** April 06, 2025

📘 **Course:** SE445: Generative AI

 **by Anis Koubaa**

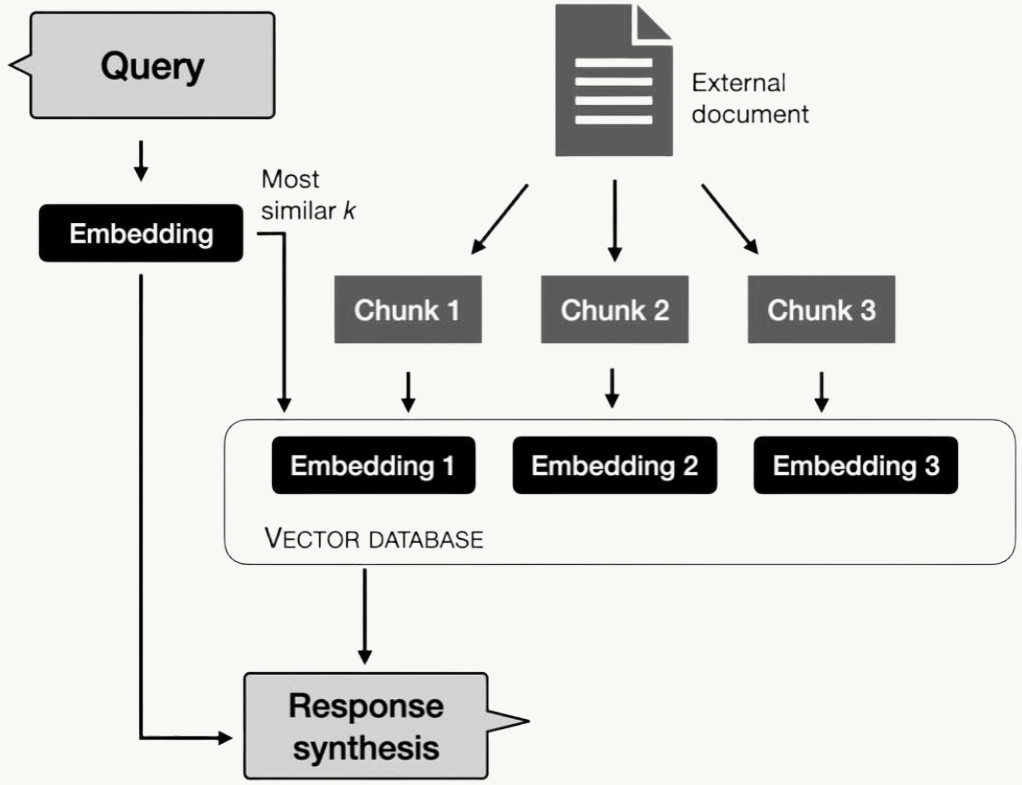
What is Chunking and Why is it Important?

Definition:

- Chunking divides large documents into smaller segments to enable efficient processing in NLP tasks.

Importance:

- **Enhanced Processing:** Smaller chunks allow models to handle information more effectively, especially with lengthy documents.
- **Improved Retrieval:** In RAG systems, chunking enables precise matching between queries and relevant document sections.
- **Context Preservation:** Proper chunking maintains semantic integrity, ensuring each chunk contains coherent information.



Key Chunking Techniques in LangChain

1 📄 Text Character Splitting

Splits the text based on a fixed character count. Uses simple **separators** like newline \n or space " ". Allows setting a **chunk overlap** to preserve context across segments.

🧪 Example: Split a document into **1,000-character chunks** with a **200-character overlap** to ensure continuity.

2 🔄 Recursive Character Splitting

Applies a **hierarchy of separators** (e.g., paragraphs → sentences → words) to split text. If the first split (e.g., paragraphs) results in large chunks, it **recursively** splits into smaller units. Helps maintain **semantic coherence** while ensuring chunk size limits.

🧪 Example: First split by paragraphs. If a paragraph is too long, then split it into sentences.

3 🌐 HTML Section Splitter

Designed for structured web content. Splits text based on **HTML tags** like h1, h2, etc. Maintains the document's **visual and logical structure** in the chunks.

4 💻 Code Splitter

Specialized for handling **programming code** files. Splits files into meaningful units such as **functions**, **classes**, or **modules**. Ensures that **code logic** remains intact in each chunk for better understanding or processing.

What is Semantic Chunking?

🧬 Definition

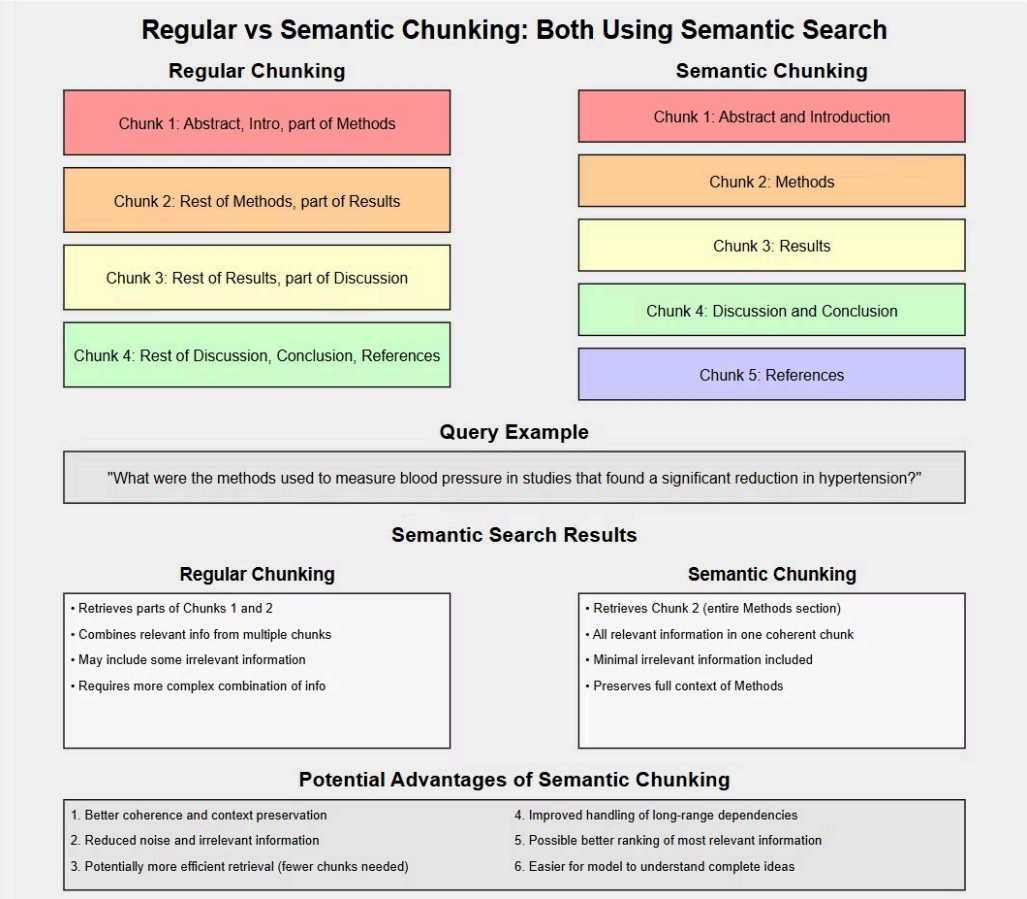
Semantic chunking divides text along **meaningful content boundaries** using **natural language understanding** to maintain contextual integrity and topical coherence.

🔍 How It Works

- Employs **sentence embeddings** to identify **topic shifts**
- Groups content with **similar meaning**
- Mimics **human comprehension** patterns

📌 Why It Matters

- Preserves **complete ideas** within chunks
- Enhances **retrieval accuracy** in RAG systems
- Minimizes **semantic drift** between chunks



Semantic Chunking: Improving AI Information Retrieval

Semantic Chunking in Practice

⚙️ Implementation Approaches

1. **Embedding Clustering:**
   - Convert sentences to embeddings → Group by similarity (e.g., k-means or hierarchical clustering).
2. **Topic Modeling:**
   - Use models like LDA or BERTopic to detect topic boundaries.
3. **Windowed Similarity:**
   - Slide a window over the text and measure semantic similarity to split intelligently.

📌 Pros

- Preserves context naturally
- Ideal for documents with **complex structure** or **rich semantics**

⚠️ Cons

- Requires more **computation** (embedding + clustering)
- Less deterministic than rule-based splitters

💡 Best Use Cases

- Research papers, legal texts, and dense technical documents
- When **retrieval accuracy** and **contextual coherence** are critical

LangChain Chunking Examples

1. **Fixed-Size (in Characters) Overlapping Sliding Window**
   - Divides text into character-based chunks with overlaps, simple to implement but may cut across sentences without semantic consideration.
   - **Example (LangChain)**:

```python
from langchain.text_splitter import CharacterTextSplitter
text_splitter = CharacterTextSplitter(chunk_size=256, chunk_overlap=20)
docs = text_splitter.create_documents([text])
```

2. **Recursive Structure Aware Splitting**
   - Balances fixed-size chunks with linguistic boundaries, maintaining context while adding implementation complexity.
   - **Example (LangChain)**:

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_size=256, chunk_overla
docs = text_splitter.create_documents([text])
```

LangChain Chunking Examples

3. **Structure Aware Splitting (by Sentence, Paragraph)**
    - Uses natural language units for chunking, preserving semantic integrity, though complex structures may vary.
    - **Example**:

```python
docs = text.split(".")
```

4. **Content-Aware Splitting (Markdown, LaTeX, HTML)**
    - Adapts to content types, maintaining integrity in structured documents but unsuitable for unstructured content.
    - **Example for Markdown (LangChain)**:

```python
from langchain.text_splitter import MarkdownTextSplitter
markdown_splitter = MarkdownTextSplitter(chunk_size=100, chunk_overlap=0)
docs = markdown_splitter.create_documents([markdown_text])
```

LangChain Chunking Examples

5. **NLP Chunking: Tracking Topic Changes**
    - Divides text based on topic shifts for semantic continuity, requiring advanced NLP techniques.
    - **Example using NLTK in LangChain:**

python Copy code

```python
from langchain.text_splitter import NLTKTextSplitter
text_splitter = NLTKTextSplitter()
docs = text_splitter.split_text(text)
```

LangChain Chunking Examples

5. **NLP Chunking: Tracking Topic Changes**
    - Divides text based on topic shifts for semantic continuity, requiring advanced NLP techniques.
    - **Example using NLTK in LangChain:**

python Copy code

```python
from langchain.text_splitter import NLTKTextSplitter
text_splitter = NLTKTextSplitter()
docs = text_splitter.split_text(text)
```